

AskDocs AI

Project Document - RAG Document Assistant

Next.js + FastAPI + LlamaIndex + Qdrant

Obiettivo: costruire un progetto semplice, con solo 4 schermate, per imparare bene RAG, embeddings, Qdrant e LlamaIndex senza aggiungere PostgreSQL, auth, billing o storage complessi.

Decisione chiave: Qdrant Cloud per il database vettoriale in produzione; Qdrant Docker in locale.

1. Obiettivo del progetto

AskDocs AI è una web app semplice che permette di caricare documenti e fare domande sul loro contenuto usando una pipeline RAG. L'obiettivo non è creare un SaaS complesso, ma un progetto portfolio per imparare bene le parti AI che ti mancano.

- RAG
- Embeddings
- Qdrant
- LlamaIndex
- FastAPI come AI backend
- Vector search
- AI answers with sources

2. Descrizione breve da portfolio

AskDocs AI is a lightweight RAG document assistant built with Next.js, FastAPI, LlamaIndex and Qdrant. It allows users to upload documents, generate embeddings, store them in a vector database and ask questions grounded in retrieved document chunks.

3. Stack tecnico

Area	Tecnologie
Frontend	Next.js, React, TypeScript, Tailwind CSS, shadcn/ui opzionale
Backend	Python, FastAPI, LlamaIndex
AI / Vector Search	OpenAI API oppure Claude API, embeddings, Qdrant, RAG pipeline
Storage	File locale in /uploads, metadata salvati dentro Qdrant payload

Non usare in questa versione: PostgreSQL, Supabase, S3, auth, billing, ruoli utente. Il progetto deve restare focalizzato su Qdrant e RAG.

4. Funzionalità principali

Il progetto avrà solo 4 schermate: Upload Document, Documents List, Chat with Document, Sources / Chunks Used.

4.1 Upload Document

Pagina dove l'utente carica un documento.

- Document title

- File upload: .txt oppure .pdf
- Button: Process document

File upload

- > Extract text
- > Split text into chunks
- > Generate embeddings
- > Save vectors + metadata in Qdrant

Metadata da salvare in Qdrant:

```
{
  "document_id": "doc-uuid",
  "document_title": "Contract",
  "filename": "contract.pdf",
  "chunk_index": 1,
  "text": "The client agrees to pay within 30 days...",
  "created_at": "2026-07-06T13:00:00Z"
}
```

4.2 Documents List

Pagina con i documenti caricati. La lista puo essere ricostruita leggendo i payload da Qdrant e raggruppando per document_id.

- Title
- Filename
- Created at
- Chunks count
- Status
- Open Chat button
- Delete button

Documents

```
[Ready] Contract.pdf    42 chunks    Open Chat
[Ready] Meeting Notes.txt 18 chunks    Open Chat
[Ready] CV.pdf          25 chunks    Open Chat
```

4.3 Chat with Document

Pagina dove l utente fa domande su un documento specifico.

- What is this document about?
- Summarize the key points.
- What are the main risks?
- Are there any deadlines?
- What obligations are mentioned?

- What are the payment terms?

User question

- > Generate question embedding
- > Search similar chunks in Qdrant
- > Filter by document_id
- > Build RAG prompt with retrieved chunks
- > Send prompt to LLM
- > Return answer

4.4 Sources / Chunks Used

Dopo ogni risposta, mostra i chunk usati. Questa schermata dimostra che il progetto usa retrieval + grounding, non solo una chiamata generica all AI.

Answer:

The document says that the client must pay within 30 days from the invoice date.

Sources used:

1. Contract.pdf - chunk 8

"The invoice must be paid within 30 days..."

2. Contract.pdf - chunk 12

"Late payments may result in additional fees..."

5. Architettura

Next.js Frontend

|

| REST API

v

FastAPI Backend

|

| extract text

| chunk text

| generate embeddings

v

Qdrant Vector Database

|

| similarity search

| metadata filtering

v

LLM API

|

v

Answer + sources

6. Pipeline RAG

Document ingestion:

```
PDF/TXT
-> Text extraction
-> Chunking
-> Embeddings generation
-> Qdrant upsert
```

Question answering:

```
Question
-> Question embedding
-> Qdrant search
-> Retrieve top-k chunks
-> Build context
-> LLM answer
-> Return sources
```

7. Qdrant design

Nome collection consigliato: askdocs_chunks. Ogni punto in Qdrant rappresenta un chunk del documento.

```
{
  "id": "chunk-uuid",
  "vector": [0.123, -0.442, 0.981],
  "payload": {
    "document_id": "doc-uuid",
    "document_title": "Contract",
    "filename": "contract.pdf",
    "chunk_index": 1,
    "text": "Chunk content here...",
    "created_at": "2026-07-06T13:00:00Z"
  }
}
```

Quando l'utente chiede qualcosa su un documento, Qdrant deve cercare solo nei chunk di quel documento:

```
search vector similar to question
where document_id = selected_document_id
limit 5
```

Questo ti fa imparare una parte importante: similarity search + metadata filtering.

8. API FastAPI

Endpoint minimi:

```
POST /documents/upload
GET /documents
POST /documents/{document_id}/ask
DELETE /documents/{document_id}
```

Upload document:

```
POST /documents/upload

Request: multipart/form-data
- title
- file

Response:
{
  "document_id": "doc-uuid",
  "title": "Contract",
  "filename": "contract.pdf",
  "status": "ready",
  "chunks_count": 42
}
```

Ask question:

```
POST /documents/{document_id}/ask

Request:
{
  "question": "What are the payment terms?"
}

Response:
{
  "answer": "The payment terms require the client to pay within 30 days from the invoice date.",
  "sources": [
    {
      "document_title": "Contract",
      "filename": "contract.pdf",
      "chunk_index": 8,
      "text": "The invoice must be paid within 30 days..."
    }
  ]
}
```

9. Struttura cartelle

Frontend:

```
frontend/
  app/
  page.tsx
```

```
upload/  
  page.tsx  
documents/  
  page.tsx  
documents/[id]/  
  page.tsx  
components/  
  UploadForm.tsx  
  DocumentCard.tsx  
  DocumentsList.tsx  
  ChatBox.tsx  
  SourcesPanel.tsx  
lib/  
  api.ts
```

Backend:

```
backend/  
  app/  
    main.py  
  api/  
    documents.py  
  services/  
    document_service.py  
    rag_service.py  
    qdrant_service.py  
    embedding_service.py  
  utils/  
    file_loader.py  
    text_splitter.py  
  config.py  
  uploads/  
  requirements.txt  
  Dockerfile  
  docker-compose.yml
```

10. MVP development plan

Step 1 - Backend only:

- Setup FastAPI
- Setup Qdrant locally with Docker
- Create Qdrant collection
- Upload .txt file
- Extract text
- Split text into chunks
- Generate embeddings
- Save chunks in Qdrant
- Ask a question

- Retrieve chunks from Qdrant
- Generate answer with LLM
- Return answer + sources

Step 2 - Frontend:

- Upload document page
- Documents list page
- Chat page
- Sources panel

Step 3 - PDF support:

- Add PDF text extraction
- Handle failed uploads
- Show processing status

11. Docker Compose locale

Per lo sviluppo locale usa Qdrant con Docker:

```
services:
  qdrant:
    image: qdrant/qdrant
    ports:
      - "6333:6333"
    volumes:
      - qdrant_storage:/qdrant/storage

volumes:
  qdrant_storage:
    QDRANT_URL=http://localhost:6333
```

12. Deploy del database vettoriale

Scelta consigliata: Qdrant Cloud. Per questo progetto non conviene fare self-hosting del database vettoriale: usa il servizio managed ufficiale, così eviti server, storage, backup e configurazione manuale.

Qdrant Cloud ha un Free Tier adatto a testing e prototipi. Dai riferimenti ufficiali Qdrant, il free tier include 1 GB RAM e 4 GB disk, senza alta disponibilità, quindi è perfetto per un progetto portfolio.

```
Frontend: Vercel
Backend FastAPI: Google Cloud Run
Vector DB: Qdrant Cloud
```

Deploy architecture:

```
Vercel
Next.js frontend
|
v
Google Cloud Run
FastAPI backend container
|
v
Qdrant Cloud
vector database
|
v
OpenAI / Claude API
LLM + embeddings
```

13. Environment variables

Frontend:

```
NEXT_PUBLIC_API_URL=https://your-fastapi-backend.com
```

Backend:

```
OPENAI_API_KEY=your_openai_key
QDRANT_URL=https://your-qdrant-cluster-url
QDRANT_API_KEY=your_qdrant_api_key
QDRANT_COLLECTION=askdocs_chunks
```

14. Dove fare deploy, in pratica

Qdrant: crea un cluster free/prototype su Qdrant Cloud. Ti serviranno Cluster URL, API Key e Collection name.

Backend FastAPI: usa Google Cloud Run. Crei un container Docker per FastAPI, lo pubblichi su Google Cloud e configuri le variabili ambiente per Qdrant e OpenAI/Claude.

Frontend: usa Vercel, perche e la scelta piu naturale con Next.js.

15. Cosa scrivere nel CV

Versione breve:

Built AskDocs AI, a lightweight RAG document assistant using Next.js, FastAPI, LlamaIndex and Qdrant. Implemented document ingestion, embeddings generation, semantic retrieval and AI answers with cited sources.

Versione piu tecnica:

Designed and implemented a full-stack RAG pipeline with Python/FastAPI, Qdrant vector database and LlamaIndex. Built document upload, chunking, vector indexing, metadata filtering, semantic search and context-aware answer generation.

Versione LinkedIn:

I built AskDocs AI to explore RAG workflows and vector databases. The app allows users to upload documents, ask questions and receive AI-generated answers grounded in retrieved sources using FastAPI, Qdrant and LlamaIndex.

16. Priorita finale

1. FastAPI backend
2. Qdrant locale con Docker
3. Upload .txt
4. Chunking
5. Embeddings
6. Save to Qdrant
7. Ask endpoint
8. Retrieve chunks
9. LLM answer
10. Sources
11. Next.js UI
12. PDF upload
13. Deploy Qdrant Cloud
14. Deploy backend Google Cloud Run
15. Deploy frontend Vercel

Questa e la versione giusta: piccola, mirata, senza tecnologie inutili, ma forte da CV.

17. Riferimenti ufficiali

- Qdrant Pricing: <https://qdrant.tech/pricing/>
- Qdrant Cloud cluster creation: <https://qdrant.tech/documentation/cloud/create-cluster/>